Les Tableaux Unidimensionnels et le Passage par Adresse en Langage C

Thème principal : Manipulation des tableaux à une dimension, contiguïté mémoire et pointeurs

Synthèse exécutive : Ce cours aborde la gestion des tableaux unidimensionnels en langage C, leur disposition contiguë en mémoire vive et l'accès direct aux éléments en temps constant O(1). Il détaille également le mécanisme fondamental de l'Array Decay lors de la transmission de tableaux aux fonctions, imposant le passage explicite de la taille en paramètre et permettant une modification directe des données originales.

Définition, allocation contiguë et calcul d'adresse mémoire

En langage C, la déclaration d'un tableau statique (par exemple int tab[5];) indique au compilateur de réserver un espace continu en mémoire vive d'une taille égale au nombre d'éléments multiplié par la taille du type de données (5 * sizeof(int)). L'accès à un élément via l'opérateur crochet tab[i] n'est pas une recherche séquentielle mais un calcul d'adresse direct : Adresse(tab[i]) = Adresse_Base + (i * sizeof(type)). C'est pourquoi le premier élément correspond à l'indice 0 (décalage nul). Cette approche garantit une complexité algorithmique en temps constant O(1).

- Tableau unidimensionnel : séquence homogène de données de même type de taille fixe
- Contiguïté mémoire : blocs alloués les uns à la suite des autres sans espace
- Indexation : positionnement démarrant obligatoirement à l'indice 0 jusqu'à N-1
- Accès en O(1) : calcul mathématique direct de l'emplacement mémoire

Cas concret enseignant : *Le professeur a comparé le tableau à un meuble de rangement vertical composé de tiroirs rigoureusement identiques et collés les uns aux autres, numérotés de 0 à N-1. Connaissant l'emplacement de base du premier tiroir et la hauteur d'un tiroir, on atteint instantanément le tiroir i sans avoir à ouvrir les précédents.*

Alignement, pointeurs et mécanisme d'Array Decay

Lorsqu'un tableau est passé en paramètre à une fonction, le langage C effectue une conversion implicite appelée Array Decay. Le nom du tableau se dégrade en un pointeur vers son premier élément (&tab;[0]). Cela évite de copier inutilement l'intégralité des données en mémoire, garantissant d'excellentes performances. En contrepartie, la fonction ne connaît plus la taille du tableau : il est donc obligatoire de passer la taille en paramètre distinct. Toute modification effectuée dans la fonction se répercute directement sur le tableau d'origine.

- Array Decay : conversion automatique du nom du tableau en pointeur vers son premier élément
- Passage par adresse : transmission de l'emplacement mémoire au lieu d'une copie
- Perte de taille : impossibilité pour la fonction appelée de connaître la taille du tableau de manière native
- Arithmétique des pointeurs : équivalence entre tab[i] et *(tab + i)

Cas concret enseignant : *Pour illustrer l'Array Decay et le passage par adresse, l'enseignant a expliqué que transmettre un tableau à une fonction ne consiste pas à construire une réplique exacte de sa maison dans le jardin de son ami (passage par valeur), mais à lui donner les coordonnées GPS de sa propre maison (passage par adresse) afin qu'il vienne modifier directement l'original.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante ou affiche des valeurs absurdes lorsque j'essaie de lire ou d'écrire dans tab[5] alors que mon tableau a été déclaré avec une taille de 5 (int tab[5];) ?
Réponse du professeur : En C, les indices d'un tableau de taille N vont de 0 à N-1. Pour un tableau de 5 éléments, les indices valides sont 0, 1, 2, 3 et 4. Accéder à tab[5] cible une zone mémoire hors limites (debordement de tampon ou buffer overflow), ce qui provoque un comportement indéfini, un plantage ou la modification accidentelle d'autres variables.

Question étudiant : Est-ce qu'on peut calculer la taille d'un tableau à l'intérieur d'une fonction avec l'expression sizeof(tab) / sizeof(tab[0]) ?
Réponse du professeur : Non, cela ne fonctionne pas à l'intérieur d'une fonction. En raison du phénomène d'Array Decay, le paramètre reçu par la fonction est un simple pointeur et non un tableau statique. L'opérateur sizeof renverra la taille du pointeur (par exemple 8 octets sur une architecture 64 bits) et non la taille totale du tableau. C'est pour cette raison impérative qu'il faut toujours passer la taille du tableau en paramètre additionnel.

■ Exercice & Démonstration d'Application

Énoncé : Écrire en C une fonction nommée 'remplir_et_calculer_moyenne' qui prend en paramètres un tableau d'entiers et sa taille effective. La fonction doit demander à l'utilisateur d'entrer chaque note au clavier pour remplir le tableau, puis calculer et retourner la moyenne arithmétique exacte des notes saisies sous la forme d'un nombre à virgule (double).

Résolution pas à pas :
1. Définir le prototype de la fonction : double remplir_et_calculer_moyenne(int tab[], int taille); 2. Ajouter une vérification de sécurité pour s'assurer que taille > 0 et éviter une division par zéro. 3. Déclarer une variable accumulateur double somme = 0.0; 4. Parcourir le tableau à l'aide d'une boucle for de i = 0 jusqu'à i < taille. 5. Demander la saisie de chaque élément via scanf("%d", &tab;[i]); et cumuler la valeur dans somme. 6. Retourner la division de la somme par la taille convertie en double.

Résultat : #include double remplir_et_calculer_moyenne(int tab[], int taille) { if (taille <= 0) { return 0.0; } double somme = 0.0; for (int i = 0; i < taille; i++) { printf("Saisir l'élément [%d] : ", i); scanf("%d", &tab;[i]); somme += tab[i]; } return somme / taille; }

■ Auto-Évaluation Flash d'Examen

Q : Quel est l'indice du dernier élément accessible d'un tableau d'entiers déclaré par int t[10]; ?

R : L'indice 9 (les indices vont de 0 à N-1, soit 0 à 9).

Q : Que contient exactement le nom d'un tableau lorsqu'il est employé seul sans crochets dans un programme C ?

R : Il contient l'adresse mémoire du premier élément du tableau, équivalent à &tab;[0].

Q : Pourquoi la signature d'une fonction manipulant un tableau C doit-elle obligatoirement inclure la taille du tableau en paramètre ?

R : À cause de l'Array Decay : lorsqu'un tableau est transmis à une fonction, il se dégrade en pointeur et perd toute information sur sa taille initiale.